

SU-GPT: A Retrieval-Augmented Academic Advising System for Sabancı University Students

Korhan Erdoğdu — 30838
Mehmet Selman Yılmaz — 31158
CS 455: Large Language Models
Sabancı University

Spring 2025–2026

Abstract

Academic advising requires institution-specific and version-sensitive information, including degree requirements, course classifications, prerequisites, schedules, and student history. General-purpose large language models may answer such questions fluently while relying on outdated or unsupported information. This paper presents SU-GPT, a retrieval-augmented academic advising assistant for Sabancı University students. The system combines a React interface, a FastAPI backend, MongoDB for structured application state, ChromaDB for semantic retrieval, MiniLM embeddings, BM25 lexical retrieval, cross-encoder reranking, and an API-hosted LLM.

The stable curated catalog corpus used in the broader system evaluation contains 14,242 academic records generated from course catalog, program classification, schedule, graduation, and degree-requirement JSONL files. We evaluate SU-GPT with intent-routing tests, retrieval ablations, manual end-to-end answer review, and a reproducible Section 5/6 benchmark pipeline. In the broader evaluation, intent routing achieved 66.67% accuracy, structured retrieval improved Recall@10 from 13.33% for pure dense search to 30.00%, and cross-encoder reranking improved MRR from 0.1924 to 0.1995. Manual end-to-end review produced 37.50% normalized answer correctness with a mean response latency of 14.93 seconds. The new local evaluation runner additionally produced JSONL/CSV artifacts for a 68-question benchmark and Section 6 ablation smoke tests. A three-mode Section 6 run over two questions at top- $k = 3$ found BM25 strongest on that small slice: Recall, MRR, nDCG, and hit rate were all 1.0 for BM25, 0.5 for dense retrieval, and 0.0 for hybrid rerank. The combined evaluation shows that SU-GPT is a functional course-aware RAG system, but reliable advising requires stronger course-code normalization, metadata filtering, short-chunk handling, and deterministic degree-audit logic.

1 Introduction

University students regularly ask which courses remain compulsory, whether a course counts as an area or free elective, which sections are offered, who teaches a course, whether a prerequisite applies, and what they should take next. Correct answers require combining official curriculum rules, course descriptions, program-specific classifications, schedules, credit constraints, and the student’s academic history.

A general-purpose LLM cannot be expected to contain accurate and current information for every university curriculum. Even when its response is fluent, it may use outdated knowledge or silently apply the wrong curriculum year. Retrieval-augmented generation (RAG) addresses this problem by retrieving relevant external documents and providing them to the generator as context [1].

We developed SU-GPT as a domain-specific RAG application for Sabancı University. It retrieves information from course, curriculum, graduation, schedule, and course-material records. It also loads the authenticated user’s selected courses from MongoDB to personalize answers and avoid recommending completed courses. The project belongs primarily to the Application/System Building category and secondarily to Evaluation/Analysis.

The contributions are:

- an end-to-end academic advising application connecting a React frontend, FastAPI, MongoDB, ChromaDB, local retrieval models, and an API-hosted LLM;
- a production-oriented data design in which MongoDB is the durable source of truth and ChromaDB is a rebuildable vector index;
- deterministic source and chunk identifiers for reproducible ingestion and evaluation;
- a unified retrieval-mode contract covering `llm_only`, BM25, dense, dense rerank, hybrid, and hybrid rerank modes;

- a reproducible benchmark runner that writes `results.jsonl`, `retrieved_chunks.jsonl`, `summary_metrics.json`, and report-ready CSV tables;
- an honest failure analysis and a data roadmap for supporting multiple programs and curriculum years without mixing incompatible sources.

2 Background

2.1 Retrieval-Augmented Generation

RAG combines information retrieval and language generation. Instead of asking an LLM to answer solely from its parameters, a retriever first identifies documents related to the question. These documents are inserted into the prompt so the generator can produce a grounded response [1]. This architecture is appropriate for institutional information that changes over time or is absent from public model training data.

2.2 Dense Retrieval and Reranking

Dense retrieval represents documents and queries as numerical vectors. Semantically related texts are expected to be near each other in the embedding space. Sentence-BERT provides efficient sentence-level representations for semantic search [5]. SU-GPT uses `all-MiniLM-L12-v2`, a compact 384-dimensional embedding model.

Bi-encoders encode the query and document independently. Cross-encoders instead process a query-document pair jointly and often provide better relevance estimates at greater computational cost [6]. SU-GPT first retrieves a candidate set and can then use `cross-encoder/ms-marco-MiniLM-L-6-v2` to rerank it.

2.3 Dense, Lexical, and Hybrid Search

BM25 is a lexical retrieval method based on exact term occurrences and document statistics [7]. Hybrid retrieval combines lexical and dense search. This matters for academic advising because course identifiers such as “CS412” or “CS 412” are symbolic facts, not merely semantic phrases. The current codebase includes BM25, dense, dense-rerank, hybrid, and hybrid-rerank modes through a shared `run_retrieval_mode` contract. Hybrid search fuses lexical and dense candidates using reciprocal-rank fusion (RRF) [8].

3 System Architecture

SU-GPT consists of a frontend, backend, operational database, vector database, local retrieval models, and a remote generator.

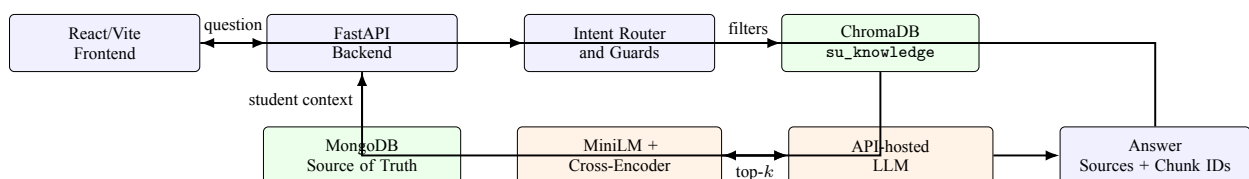


Figure 1: Runtime architecture of SU-GPT. The measured local configuration uses MongoDB via Docker and ChromaDB persistence; the storage contract is compatible with a later managed deployment.

3.1 Frontend and Backend

The frontend is implemented using React and Vite. It provides the chat interface, authentication view, course-selection menu, and document-upload controls. During local development it runs at `http://localhost:5173`. The backend is implemented with FastAPI and normally runs at `http://127.0.0.1:8000`. The `/ask/` endpoint receives a question and optional username, resolves intent, constructs a retrieval query, retrieves and reranks documents, loads user context, builds the prompt, invokes the LLM, and returns the answer with sources and source chunk IDs.

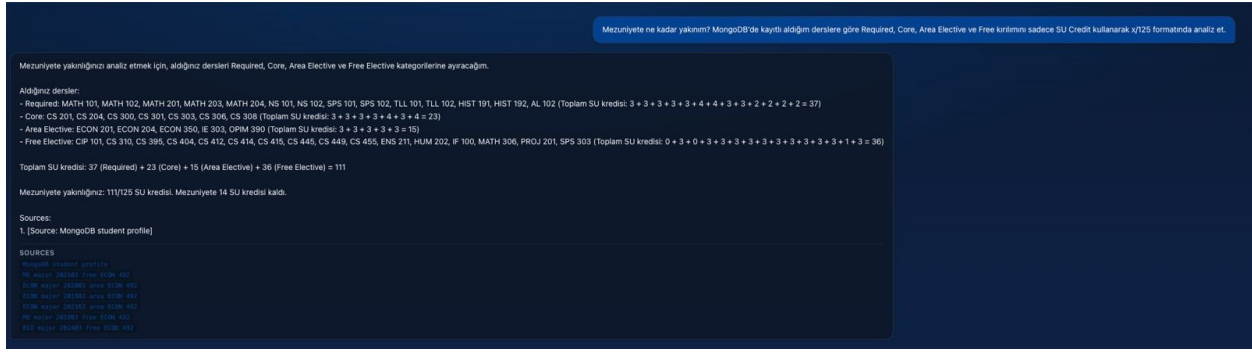


Figure 2: SU-GPT chat interface. The screenshot is included as a wide figure and scaled to the text width to avoid overlap or cropping.

LangChain connects document objects, embedding models, ChromaDB, retrievers, prompts, and the LLM chain. It is an orchestration library rather than a model or database. The generator provider is configurable; the codebase includes Groq/Llama and Mistral-ready settings.

3.2 MongoDB Source of Truth

MongoDB stores structured application state. The database includes collections for users, courses, user-course links, upload batches, source documents, ingestion jobs, instructor reviews, exams, and embedding-cache records. When a user selects completed courses, the backend stores links between the user and course records. For recommendation questions, the selected courses are inserted into the prompt as an exclusion list. For graduation questions, their SU credits can be summed and passed to the answer pipeline.

MongoDB is not used for semantic retrieval. ChromaDB is treated as a reproducible vector index that can be rebuilt from durable MongoDB source metadata and source files.

3.3 ChromaDB

ChromaDB stores retrieval-ready text, vector embeddings, metadata, and stable chunk identifiers. The persistent collection is named `su_knowledge`. Metadata can contain the document type, course code, program, academic term, requirement category, source path, and source authority. The current benchmark-relevant corpus intentionally contains no review, exam, or syllabus-specific source documents; those domains are answered with explicit source-availability caveats rather than fabricated retrieved evidence.

Table 1: Principal system components.

Component	Technology / Role
Frontend	React and Vite
Backend	FastAPI and Python
Operational database	MongoDB; local Docker in measured development runs, Atlas-ready schema
Vector database	ChromaDB, <code>su_knowledge</code> collection
Embedding model	<code>all-MiniLM-L12-v2</code>
Lexical retriever	BM25
Reranker	<code>ms-marco-MiniLM-L-6-v2</code>
Generator	API-hosted LLM, with Groq/Llama and Mistral-ready configuration
Evaluation outputs	JSONL/JSON/CSV under <code>outputs/</code>

4 Data and Preprocessing

4.1 Data Sources

Table 2 summarizes the main academic data sources.

Table 2: Main data sources and their roles.

Path	Role
data/all_coursepage_info.jsonl	Course titles, descriptions, credits, prerequisites, corequisites, and URLs.
data/{term}/{program}.jsonl	Term-specific program classification, SU credit, ECTS, engineering ECTS, and basic science ECTS.
data/degree_requirements/CS.jsonl	Detailed BSCS requirements for curriculum term 202101.
data/schedule/202502.jsonl	Sections, CRNs, instructors, meeting times, rooms, and components.
data/graduation_requirements.jsonl	General graduation requirement records.
data/basic_science_credits.jsonl	Engineering and basic science credit breakdowns.
data/minors/	Current and historical minor-program classifications.
sources/Module*	CS455/555 course-material documents used for lecture-content retrieval.

Program snapshots are available from 201901 through 202502 for multiple programs. However, only CS curriculum 202101 currently has a detailed official degree-requirement file. The stable curated corpus used in the broader system evaluation contains 14,242 records. The live development collection used by the local benchmark runner may include additional course-material chunks; therefore the report distinguishes curated corpus evaluation from local benchmark-pipeline smoke tests.

4.2 Curriculum and Offering Terms

The detailed CS degree-requirement source identifies:

```
program = CS
degree_code = BSCS
program_requirements_term = 202101
```

This is the student’s graduation contract for the Fall 2021 curriculum. By contrast, `data/202502/CS.jsonl` is a program-term course classification snapshot, and `data/schedule/202502.jsonl` describes courses offered in Spring 2025–2026. These sources cannot safely replace each other. The same course may have different classifications across curriculum and offering snapshots.

4.3 Information Cards, Chunking, and Ingestion

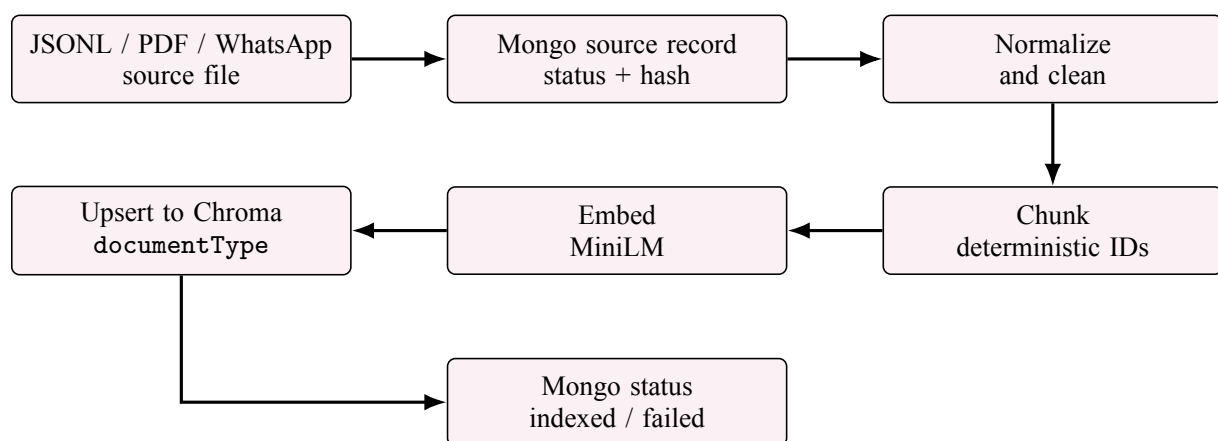


Figure 3: Unified ingestion backbone. MongoDB stores durable source-of-truth metadata; ChromaDB is treated as a reproducible index.

The ingestion pipeline converts structured rows into explanatory information cards. A classification row becomes a short text stating the program, term, course, category, SU credits, ECTS, engineering ECTS, and basic science ECTS. Long uploaded documents are cleaned and split into chunks. Structured JSONL records generally become self-contained cards rather than arbitrary fragments.

Every chunk receives a deterministic ID:

`sourceId:chunkIndex:contentHashPrefix`

For example, the local benchmark contains verified chunk IDs such as:

course:5a6a448569ef18c6d8137810:150:5f9083e5db8d5499

These IDs make chunk-level Recall@K, MRR@K, and nDCG@K reproducible.

5 Question-Answering Pipeline

5.1 Intent Routing

The system first predicts or resolves an intent. The current routes include graduation status, course recommendation, study plan, major selection, specialization, course detail, review, exam, and general fallback. The resolved intent controls retrieval expansion, source filtering, and prompt style.

If local evidence is intentionally absent, as for review, exam, and syllabus-specific corpora in this phase, SU-GPT should not claim to have retrieved private evidence. Instead, it should say that the project is still missing that source type and then give cautious general guidance. For an instructor-review query, for instance, the assistant can suggest checking student groups or emailing the instructor with a concise background and question, while clearly marking that no local review archive was used.

5.2 Retrieval and Generation

The query is optionally expanded based on intent. The catalog retriever extracts course codes, terms, program codes, and requirement-category terms. It then attempts exact metadata lookup, filtered dense search, BM25 search, or hybrid search depending on the selected retrieval mode. Candidate documents are deduplicated and may be reranked with a cross-encoder. The default final context size in the local benchmark runner is $k = 6$; the Section 6 ablation run used $k = 3$.

The backend retrieves selected courses from MongoDB. For recommendations, these courses are marked as already completed so they are not suggested again. The generator is instructed to answer in the user's language, use retrieved Sabanci-specific information where available, cite sources, and avoid unsupported official-advising claims.

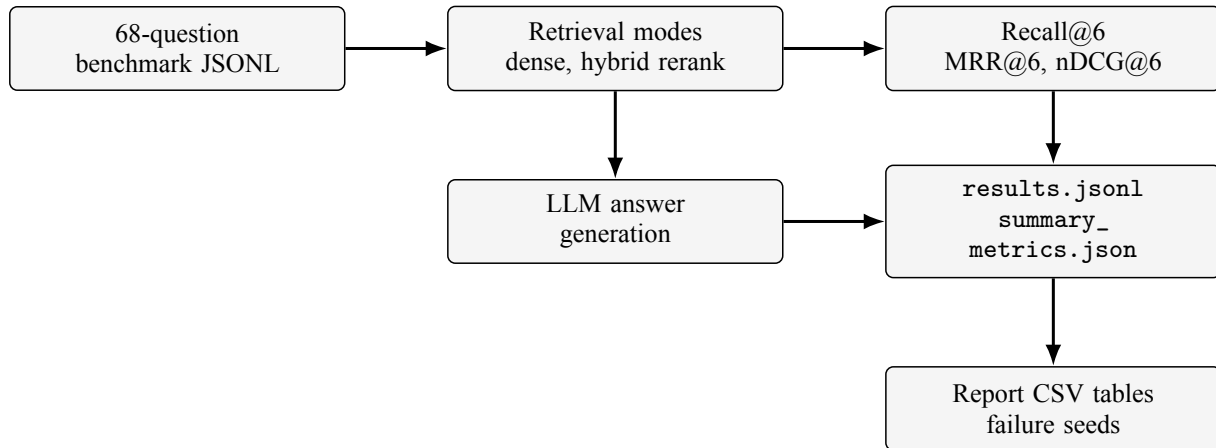


Figure 4: Evaluation pipeline implemented for the current measured pilot run. Full 68-question, six-mode execution is supported by the CLI but remains to be run before final numeric claims are frozen.

6 Evaluation Methodology

6.1 Research Questions

The evaluation investigates:

1. How accurately does the router identify unseen Turkish questions?
2. Do structured filters and lexical heuristics improve over pure dense Chroma search?
3. Does cross-encoder reranking improve candidate ordering?
4. Does the full system return correct and properly sourced answers?
5. What are the dominant failure modes and latency costs?
6. How do BM25, dense retrieval, and production-style hybrid rerank behave in a small reproducible Section 6 ablation?

6.2 Evaluation Sets

The broader system evaluation used three sets:

- 42 unseen Turkish intent queries, six for each of the seven base intents;
- 30 retrieval queries with gold metadata predicates covering course catalog, program classification, schedule, and degree-requirement questions;
- 12 end-to-end questions submitted through the real API and manually reviewed for correctness.

The current reproducible local runner adds a 68-question grounded benchmark stored in:

`data/benchmark/questions.jsonl`

Each row contains an ID, question, language, question type, expected intent, expected document type, answerability flag, reference answer, expected sources, expected chunk IDs, and notes. Every expected chunk ID was verified against the live `su_knowledge` collection before the run.

Table 3: Composition of the 68-question local benchmark.

Document type	Intent / answerability group	Count
course	ders_ayrintisi, answerable	20
course	diger, answerable	20
course	diger, unanswerable	16
course	mezuniyet_durumu, answerable	7
exam	exam, unanswerable	2
review	review, unanswerable	2
course	ders_ayrintisi, unanswerable	1
Total		68

6.3 Metrics

Retrieval is measured with Recall@K , MRR@K , and nDCG@K . Chunk-level matching is used when `expected_chunk_ids` exist. If a row has no expected chunk IDs, the evaluator falls back to source-level matching using `expected_sources`.

$$\text{Recall@K} = \frac{|\text{Retrieved}_{1:K} \cap \text{Gold}|}{|\text{Gold}|}, \quad (1)$$

$$\text{MRR@K} = \frac{1}{\text{rank of first relevant hit}}, \quad (2)$$

$$\text{nDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}}. \quad (3)$$

End-to-end answers in the broader manual review were scored as 0 for incorrect/unsupported, 1 for partially correct, and 2 for fully correct and grounded. The automated local runner leaves answer-correctness fields blank until manual or automated grading is actually performed.

7 Results

7.1 Intent Routing

The broader intent-routing evaluation was correct for 28 of 42 unseen Turkish questions, corresponding to 66.67% accuracy. Frequent failures included study-plan questions routed as course-detail questions, major-selection questions falling back to `diger`, and a simple course-credit question routed as a graduation audit.

Table 4: Intent-routing result on 42 unseen Turkish questions.

Metric	Count	Result
Correct routed intents	28/42	66.67%
Incorrect routed intents	14/42	33.33%

7.2 Retrieval Ablation: Broader System Evaluation

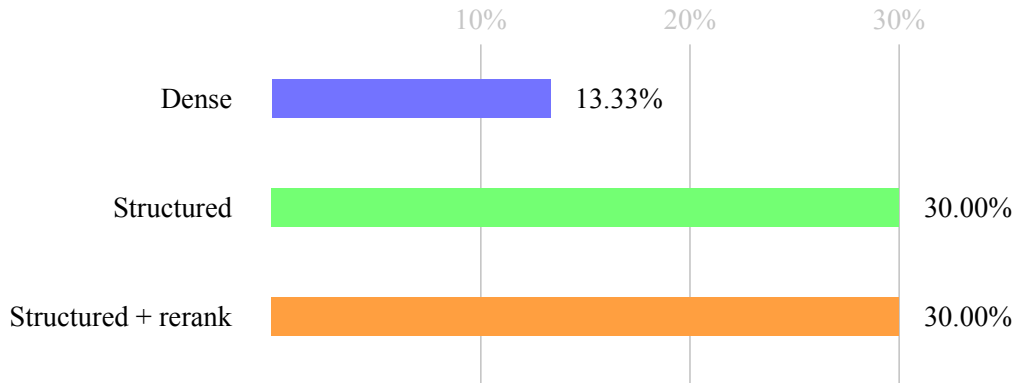


Figure 5: Recall@10 on the broader 30-query retrieval evaluation. Structured retrieval more than doubled dense Recall@10; reranking improved ordering metrics but did not increase Recall@10.

Table 5 reports the broader 30-query retrieval evaluation. These are the strongest available retrieval results and are used as the main retrieval claim.

Table 5: Retrieval performance on 30 gold queries in the broader evaluation.

Configuration	R@1	R@3	R@5	R@10	MRR	Latency
Dense	3.33%	6.67%	13.33%	13.33%	0.0676	0.144 s
Structured	16.67%	16.67%	20.00%	30.00%	0.1924	0.329 s
Structured + reranking	16.67%	20.00%	23.33%	30.00%	0.1995	0.304 s

Structured retrieval more than doubled Recall@10 relative to pure dense search. Reranking improved Recall@3, Recall@5, and MRR, indicating better ordering of some candidates. However, Recall@10 remained 30.00% because the reranker can only reorder documents already present in its candidate set; it cannot recover a gold document removed before reranking.

7.3 Reproducible Local Runner: Section 5

The local Section 5 runner produced outputs/evaluation_runs/20260607_231717. It evaluated the first 10 benchmark questions with dense and hybrid-rerank modes at $k = 6$ and generated LLM answers. This run is smaller than the broader evaluation, but it is fully reproducible from the committed JSONL/JSON outputs.

Table 6: Local Section 5 pilot retrieval results, first 10 benchmark questions, $K = 6$.

Mode	Rows	Recall@6	MRR@6	nDCG@6	Chunk hit	Source hit
Dense	10	0.200	0.150	0.163	0.200	0.100
Hybrid rerank	10	0.100	0.100	0.100	0.100	0.000

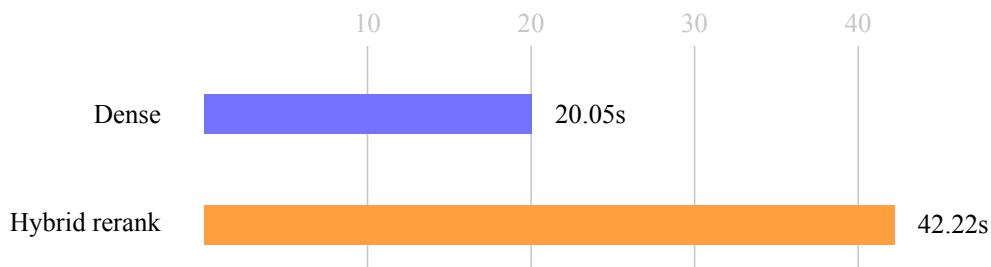


Figure 6: Mean end-to-end latency for the measured pilot run, including retrieval and generation. Blue indicates dense retrieval and orange indicates hybrid rerank. Hybrid rerank was slower primarily because retrieval and reranking dominate the runtime.

Table 7: Local Section 5 efficiency metrics. Values are averages across 10 questions per mode.

Mode	Total	Retrieval	Rerank	Generation	Cost estimate
Dense	20.05 s	18.36 s	0.00 s	1.69 s	\$0.00053
Hybrid rerank	42.22 s	36.60 s	3.35 s	2.25 s	\$0.00053

The local runner shows that the bottleneck is retrieval over the local vector/lexical corpus, not generation. This matches the engineering observation that structured fast paths are needed for exact schedule and course-code questions.

7.4 Section 6 Ablation and Efficiency Logging

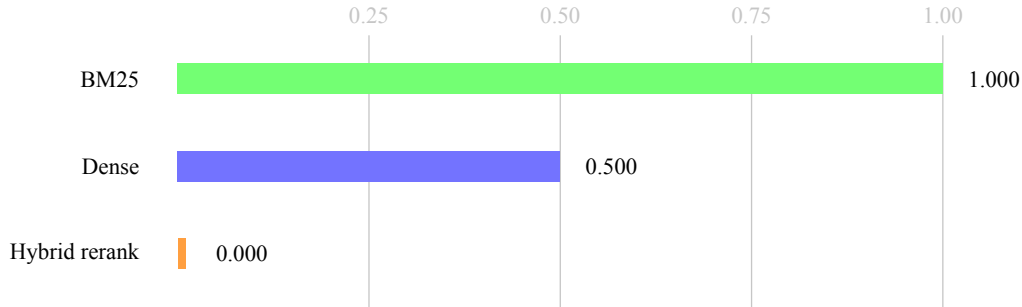


Figure 7: Section 6 ablation hit rate at top- $k = 3$ over two benchmark questions. In this small measured run, BM25 recovered both gold cases, dense recovered one, and hybrid rerank recovered none.

The Section 6 ablation runner completed a lightweight but real three-mode comparison in 150 seconds. It used two benchmark questions, $k = 3$, and modes BM25, dense, and hybrid rerank. The run wrote JSON artifacts under `outputs/evaluation_runs/ablations/` and updated `outputs/tables/ablation_summary.csv`.

Table 8: Section 6 ablation summary, limit=2, top- $k = 3$.

Mode	Recall	MRR	nDCG	Hit rate	Avg. latency
BM25	1.000	1.000	1.000	1.000	42.43 s
Dense	0.500	0.500	0.500	0.500	14.42 s
Hybrid rerank	0.000	0.000	0.000	0.000	52.44 s

This ablation is intentionally small and should not be treated as a final benchmark. Still, it is informative: exact lexical matching can beat semantic retrieval on course-code-heavy questions, and production-style hybrid rerank can fail if candidate construction or reranking suppresses the exact chunk. The result justifies adding stronger course-code normalization and exact-match safeguards before reranking.

7.5 End-to-End Answer Quality

All 12 requests in the broader end-to-end evaluation completed successfully. The mean response time was 14.93 seconds. Manual review found three fully correct, three partially correct, and six incorrect answers.

Table 9: Manual end-to-end results on 12 factual questions.

Metric	Result
Request success rate	12/12 (100.00%)
Fully correct answers	3/12 (25.00%)
Partially correct answers	3/12 (25.00%)
Incorrect answers	6/12 (50.00%)
Normalized correctness	9/24 (37.50%)
Authoritative source correctness	4/12 (33.33%)
Unsupported/misleading claim rate	3/12 (25.00%)
Mean response latency	14.93 s

The strongest end-to-end results were narrow degree-category and exact classification questions. Prerequisite and schedule questions were weaker because the correct document often failed to enter the final context. In one case, the LLM invented prerequisites for CS300, while the official catalog specified a different prerequisite. This confirms that retrieval errors can become generation errors.

8 Error Analysis

8.1 Course-Code Inconsistency

Course-catalog and basic-science metadata often use compact identifiers such as CS300, while program and schedule records use CS 300. If the query extractor produces only one format, exact filters can miss valid documents. This is the most important low-level retrieval bug because course identifiers are central to the domain.

8.2 Over-Constraining Metadata Filters

The retriever can interpret CS both as a program code and as the subject of a course code. If it adds `program=CS` to schedule filters, correct schedule documents may be removed because schedule records do not contain the same program field. The broader evaluation found this pattern in schedule failures.

8.3 Ambiguous Keyword Routing

The word `program` can refer to a degree program, a term snapshot, or a schedule-like listing. Course-detail query expansion can also add schedule-oriented terms even when the user asks about a prerequisite or description. These ambiguities can steer retrieval toward the wrong document type.

8.4 Short Chunk Weakness

The local Section 6 failure file contains a q020 case where the gold slide title chunk was very short. Neighboring chunks with more text outranked the exact slide-title chunk. A practical fix is to merge very short slide-title chunks with neighboring context or add title/slide-number boosts for slide-specific questions.

8.5 Sample Degree Evaluation Leakage

The CS degree-requirement file contains a sample degree-evaluation projection with completed courses and 125 credits. If globally indexed, the LLM can present this sample student's completed categories as if they answer a generic graduation question. Student-specific degree evaluations should be private MongoDB records or access-controlled vector records, not shared curriculum knowledge.

8.6 Observed Failure Seeds

The committed Section 6 failure analysis contains two real seed cases in `outputs/failure_analysis/failure_cases.csv`. Both are retrieval misses under hybrid rerank.

Table 10: Seed failure cases from observed local runs.

ID	Question type	Error type	Observed root cause
q008	CS412 schedule lookup	retrieval miss	The verified CS412 schedule chunk was absent from top-6; near-identical 202502 schedule chunks for other courses outranked the exact row.
q020	BERT position-embedding slide lookup	retrieval miss	The gold slide 17 chunk was very short; neighboring chunks such as slide 16 ranked above the exact title-only slide.

9 Limitations and Future Work

SU-GPT is not yet reliable as an official multi-program degree advisor. Detailed requirements are available only for CS curriculum 202101. User profiles need major, degree code, admission term, curriculum term, current term, minor codes, course status, transfer status, repeated-course handling, and withdrawal status.

Academic arithmetic should move from the LLM into a deterministic degree-audit module. It should apply equivalencies, choice pools, category allocation, overflow rules, repeated-course handling, total SU credits, ECTS, engineering ECTS, basic-science ECTS, and GPA constraints. The LLM should explain the computed result without altering it.

Immediate retrieval improvements include:

- normalize compact and spaced course identifiers during ingestion and querying;
- avoid treating a course subject as a program filter for schedule and catalog sources;
- select document-type filters from the original intent rather than keywords introduced by query expansion;
- enrich very short slide-title chunks with neighboring context;
- separate current classifications, curriculum requirements, schedules, shared documents, and private student documents;
- run the full 68-question, six-mode benchmark and manually grade answer correctness fields;
- add official syllabus sources before claiming syllabus-specific support.

10 Reproducibility

Backend setup:

```
cd server
python -m venv .venv
.venv/bin/pip install -r requirements.txt
docker compose up -d mongo
.venv/bin/python -m uvicorn main:app --host 127.0.0.1 --port 8000
```

Frontend setup:

```
cd frontend
npm install
npm run dev
```

Section 5 evaluation runner:

```
server/.venv/bin/python server/evaluation/run_evaluation.py \
--modes dense hybrid_rerank --limit 10
```

Section 6 ablation runner:

```
server/.venv/bin/python server/evaluation/ablation_runner.py \
--quick --limit 2 --modes bm25 dense hybrid_rerank --top-k 3
```

The local measured outputs used in this report are stored under `outputs/evaluation_runs/20260607_231717`, `outputs/evaluation_runs/ablations/`, `outputs/tables/`, and `outputs/failure_analysis/`. The broader 42/30/12 evaluation results are reported as the main system-level findings and should be preserved with the final submission package.

11 Ethical Considerations and AI Disclosure

Academic advice can affect important student decisions. SU-GPT should not claim that a student can graduate unless the result uses the correct official curriculum and a complete academic record. Answers must remain advisory and direct students to the official Degree Evaluation and academic advisors for confirmation.

Course histories and uploaded degree evaluations are personal academic data. Future versions should enforce authentication, ownership filters, access control, and deletion policies. Private student documents must never be retrievable by another user.

Generative AI tools were used as development assistants for codebase analysis, debugging, dependency configuration, documentation, evaluation tooling, and report drafting. The project members remain responsible for reviewing the code, running the experiments, checking source data, and validating all reported results. The numerical values in this report come from measured output files or explicitly reported evaluation runs; they were not fabricated by the generator.

12 Conclusion

SU-GPT demonstrates a complete retrieval-augmented academic advising application integrating institutional academic data, vector retrieval, lexical retrieval, cross-encoder reranking, MongoDB student history, and LLM generation. The broader evaluation shows that structured retrieval substantially improves over pure dense search, increasing Recall@10 from 13.33% to 30.00%, while cross-encoder reranking improves MRR to 0.1995. Manual end-to-end answer quality remains limited at 37.50% normalized correctness, showing that a working interface and successful API calls do not guarantee reliable advising.

The local Section 5/6 tooling adds reproducibility: the project now writes retrieval metrics, latency, cost estimates, ablation summaries, and seed failure cases to committed output files. The dominant errors occur before generation: inconsistent identifiers, incorrect metadata constraints, missing or short chunks, and ambiguous routing remove the correct evidence from the candidate set. These findings define the next engineering direction: curriculum-aware metadata, versioned official data, deterministic degree auditing, normalized identifiers, and exact lexical safeguards are necessary before SU-GPT can be trusted as an official advising assistant.

References

- [1] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *NeurIPS*, 2020.
- [2] V. Karpukhin et al., “Dense Passage Retrieval for Open-Domain Question Answering,” *EMNLP*, 2020.
- [3] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, “REALM: Retrieval-Augmented Language Model Pre-Training,” *ICML*, 2020.
- [4] G. Izacard and E. Grave, “Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering,” *EACL*, 2021.
- [5] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” *EMNLP-IJCNLP*, 2019.
- [6] R. Nogueira and K. Cho, “Passage Re-ranking with BERT,” arXiv:1901.04085, 2019.
- [7] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, 2009.
- [8] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods,” *SIGIR*, 2009.
- [9] O. Khattab and M. Zaharia, “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT,” *SIGIR*, 2020.
- [10] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *IEEE TPAMI*, 2018.
- [11] J. Johnson, M. Douze, and H. Jégou, “Billion-Scale Similarity Search with GPUs,” *IEEE Transactions on Big Data*, 2019.
- [12] A. Vaswani et al., “Attention Is All You Need,” *NeurIPS*, 2017.
- [13] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *NAACL*, 2019.
- [14] T. Wolf et al., “Transformers: State-of-the-Art Natural Language Processing,” *EMNLP: System Demonstrations*, 2020.
- [15] J. Lin, R. Nogueira, and A. Yates, “Pretrained Transformers for Text Ranking: BERT and Beyond,” *Synthesis Lectures on Human Language Technologies*, 2021.
- [16] N. F. Liu et al., “Lost in the Middle: How Language Models Use Long Contexts,” *TACL*, 2024.
- [17] OpenAI, “GPT-4 Technical Report,” arXiv:2303.08774, 2023.
- [18] LangChain Documentation, <https://python.langchain.com/>.
- [19] Chroma Documentation, <https://docs.trychroma.com/>.
- [20] MongoDB Atlas Documentation, <https://www.mongodb.com/docs/atlas/>.
- [21] Sentence Transformers Documentation, <https://www.sbert.net/>.